

# TechPulse — Full Project Spec

## What This Project Is

A real-time dashboard that surfaces trending tech news, market data, and research breakthroughs — combining live API integration, embeddings-based clustering, and LLM summarization. Not a chatbot wrapper — a genuine data pipeline + interactive dashboard.

---

## Features

1. **Trend Cards** — Live tech news pulled via API, grouped into trend clusters (multiple articles on the same story become ONE card). Each card shows a short auto-generated summary. Tap to expand for full details + sources.
2. **“Why It Matters” Blurb** — Separate LLM call per trend that explains industry impact/reasoning, not just summarizing. Shows LLM used for reasoning, not just compression.
3. **Trend Momentum Indicator** — Tracks mention-count of each trend cluster over time (fetch cycle to fetch cycle). Flags as rising ▲, falling ▼, or steady based on rate of change. NOTE: this is math/heuristic, NOT machine learning — be clear about that distinction.
4. **Similar Past Trend Retrieval** — Stores embeddings of past trend clusters. When a new trend appears, does a similarity search against history and surfaces related past trends (e.g. “resembles GPT-4 launch buzz from March”). This is a lightweight RAG (Retrieval-Augmented Generation) system.
5. **Market Cap Snapshot** — Bar chart of top tech companies by market cap, updates daily. Revenue shown as secondary stat (quarterly, since real revenue doesn’t change daily).
6. **Tech Findings Feed** — Separate section combining:
  - arXiv API → academic AI/CS papers (tagged “Research”)
  - GNews filtered by keywords (“launch”, “breakthrough”, “unveils”, “announces”) → industry/product news (tagged “Industry”)

**Explicitly dropped from scope:** sentiment tagging, anomaly detection on market cap.

Keeping scope tight.

---

## AIML Concepts Used (know these cold for interviews)

| Concept           | What it actually is                                                      | Where it's used                                                                   |
|-------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Embeddings        | Pre-trained model converts text<br>→ vector of numbers capturing meaning | Clustering + trend memory retrieval                                               |
| Cosine similarity | Measures how "close" two vectors point (range -1 to 1)                   | Grouping duplicate articles into one trend; similarity search against past trends |
| Clustering        | Grouping articles whose similarity score exceeds a threshold             | Turning 5 articles about the same story into 1 card                               |
| LLM summarization | Prompting an existing model (not training one) to compress/explain text  | Card summaries + "why it matters" blurbs                                          |
| RAG (lightweight) | Retrieving relevant past data before generating new output               | Similar past trend retrieval                                                      |
| Momentum scoring  | NOT ML — just rate-of-change math on mention counts over time            | Rising/falling/steady tags                                                        |

Be upfront in interviews: you integrated and applied pre-trained models + LLM APIs, you did not train models from scratch. That's still a legitimate and valuable skill set — just be accurate about it.

---

## Tech Stack (final, no "or")

- **Frontend:** React + Vite + Tailwind CSS + Framer Motion (same stack as NeuroFlow AI)
- **Backend: FastAPI** (Python) — chosen over Node/Express because the ML pipeline (sentence-transformers, embeddings, clustering) is Python-native; keeps everything in one language/codebase without cross-language calls

- **Storage:** SQLite (or simple JSON file store) for trend history + embeddings — no need for a full vector DB at this scale
  - **Embeddings model:** `sentence-transformers` ( `all-MiniLM-L6-v2` ) — runs locally, free, no API key
  - **LLM:** Anthropic Claude API — summarization + “why it matters” generation
  - **News API:** GNews — free tier, no credit card needed
  - **Market data:** `yfinance` (Python library) — free, no API key, no rate-limit headaches
  - **Research data:** arXiv API — free, no key needed
- 

## Final API / Library List

| Purpose           | Tool                               | Key needed?       | Notes                                  |
|-------------------|------------------------------------|-------------------|----------------------------------------|
| Tech news         | GNews API                          | Yes (free signup) | Powers trend cards + industry findings |
| Embeddings        | <code>sentence-transformers</code> | No                | Runs locally                           |
| LLM summarization | Anthropic Claude API               | Yes               | Summaries + why-it-matters             |
| Market cap        | <code>yfinance</code>              | No                | Python library, scrapes Yahoo Finance  |
| Research papers   | arXiv API                          | No                | Academic findings feed                 |

---

## Pipeline

**News/Trend Pipeline (runs every fetch cycle, e.g. every 2 hours via cron/scheduler)**

1. Fetch tech headlines (`GNews API`)
2. Generate embeddings for each article (`sentence-transformers`)

3. Cluster similar articles into trend groups (cosine similarity, threshold ~0.75)
4. Compare each new cluster against stored past trend embeddings  
→ similarity search → flag "similar to X" if match found
5. Calculate momentum: compare today's mention-count vs last cycle's for each cluster  
→ tag rising/falling/steady
6. LLM call (Claude API): generate short summary + "why it matters" line per cluster
7. Save all trend data (cluster info, summary, momentum, similar-trend reference,

## Market Data Pipeline (separate, simpler, runs daily)

1. Fetch market cap for tracked company list (yfinance)
2. Store daily values (append to history)
3. Serve latest snapshot to frontend for bar chart

## Tech Findings Pipeline (runs alongside news pipeline)

1. Fetch arXiv papers (AI/CS categories) → tag "Research"
2. Fetch GNews filtered by keywords: launch, breakthrough, unveils, announces → tag "Industry"
3. Merge into single feed, sorted by recency

## Serving Layer (FastAPI backend)

Endpoints:

- |                     |                                                       |
|---------------------|-------------------------------------------------------|
| GET /api/trends     | → trend cards (summary, momentum, similar-trend link) |
| GET /api/trend/{id} | → expanded detail view for one trend                  |
| GET /api/market     | → market cap bar chart data                           |
| GET /api/findings   | → tech findings feed (research + industry)            |

## Frontend (React)

- Fetches from FastAPI endpoints above
- Trend card grid → Framer Motion expand/collapse animation on tap
- Market cap bar chart (Recharts or D3)
- Tech findings feed section (smaller card list, badge for Research/Industry)
- Dark, cinematic theme – consistent with NeuroFlow AI visual style

---

## Suggested Folder Structure

```

techpulse/
├── backend/
│   ├── main.py                (FastAPI app entrypoint)
│   ├── services/
│   │   ├── news_fetcher.py    (GNews API calls)
│   │   ├── embeddings.py      (sentence-transformers wrapper)
│   │   ├── clustering.py      (cosine similarity grouping logic)
│   │   ├── summarizer.py      (Claude API calls for summaries + why-it-matters)
│   │   ├── momentum.py        (mention-count tracking + rate of change)
│   │   ├── trend_memory.py    (stores + retrieves past trend embeddings)
│   │   ├── market_data.py     (yfinance calls)
│   │   └── findings.py        (arXiv + keyword-filtered GNews)
│   ├── storage/
│   │   └── trends.db (SQLite) or trends_history.json
│   ├── routes/
│   │   └── api.py              (all endpoint definitions)
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── TrendCard.jsx
│   │   │   ├── MarketChart.jsx
│   │   │   └── FindingsFeed.jsx
│   │   ├── App.jsx
│   │   └── index.css (Tailwind)
│   ├── package.json
│   └── vite.config.js
└── README.md

```

## Realistic Timeline (~40-55 hrs total, 3-4 weeks part-time)

| Phase  | Work                                                                       | Est. Hours |
|--------|----------------------------------------------------------------------------|------------|
| Week 1 | News fetch + embeddings + clustering + LLM summarization (MVP, ugly UI ok) | ~12-14 hrs |
| Week 2 | Momentum scoring + trend memory/retrieval                                  | ~5-7 hrs   |
| Week 3 | Market cap section + tech findings feed + backend routes                   | ~9-12 hrs  |

|           |                                                                  |               |
|-----------|------------------------------------------------------------------|---------------|
| Week<br>4 | React frontend build, styling, animations, deployment, debugging | ~18-22<br>hrs |
|-----------|------------------------------------------------------------------|---------------|

Build in checkpoints — each week should leave you with something demoable, not a half-finished pile.

---

## Resume Bullet (draft)

"Built TechPulse, a real-time tech intelligence dashboard integrating live news, market, and research APIs with embeddings-based clustering, LLM-powered summarization, and a lightweight RAG system for historical trend retrieval; deployed with a React + FastAPI stack."

---

## Naming

**TechPulse** — short, pairs well with dark cinematic UI (pulsing glow effects, heartbeat-style animation for momentum indicator).

---

## What NOT to overclaim in interviews

- You did not train an ML model — you used pre-trained embeddings + an LLM API. Be accurate.
- Momentum scoring is a heuristic (rate-of-change math), not ML.
- Be ready to explain cosine similarity and clustering threshold choice in plain terms.